

Problema y solución

¿Qué pasa sin una BD bien diseñada?

- Productos vendidos aún visibles como disponibles
- Usuarios editando artículos ajenos
- Chats duplicados y mensajes sin orden
- Consultas lentas y datos incoherentes

Arquitectura del proyecto

El backend actúa como intermediario: el frontend nunca accede directamente a la base de datos.

Usuario

Interfaz React/Vite
que inicia solicitudes
al backend.

Backend

FastAPI expone API
REST y valida
peticiones entrantes.

Supabase

Gestión de
autenticación y
lógica de acceso a la
BD.

PostgreSQL

Almacenamiento
persistente y
consultas
optimizadas.

Modelo E/R y tablas principales

Usuarios y acceso

tabla usuarios



Catálogo

tablas articulos,
fotos_articulo



Interacción

tablas favoritos,
conversaciones,
mensajes,
ofertas

Compra y soporte



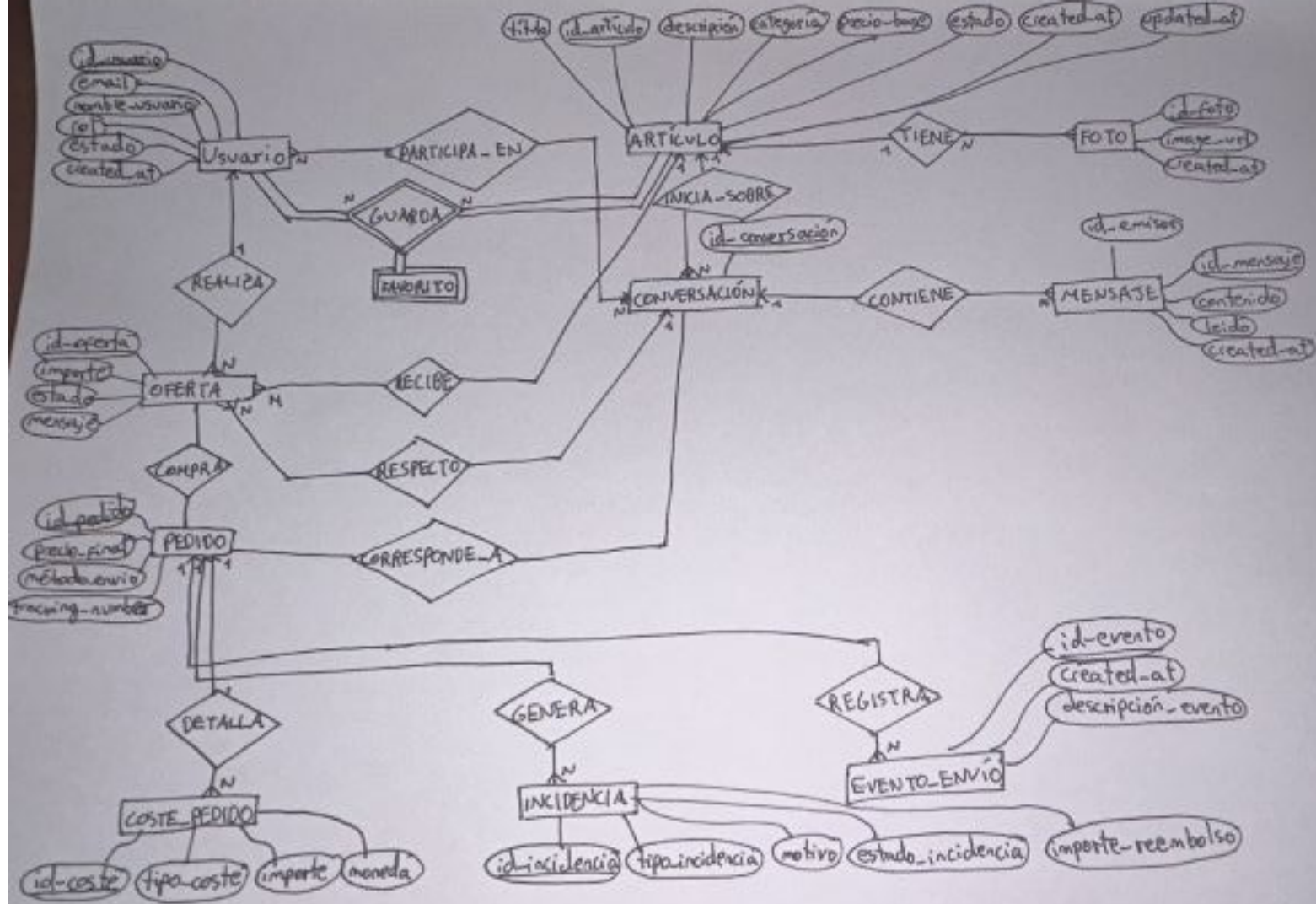
tablas pedidos,
costes_pedidos,
eventos_envio,
incidencias

¿Qué son los dominios?

Agrupaciones visuales para entender el modelo. No son bases de datos separadas ni tipos SQL.

Relaciones clave

- Un usuario publica muchos artículos
- Un artículo tiene muchas fotos
- Una conversación contiene muchos mensajes
- Un pedido puede tener costes, eventos e incidencias



Supabase: conexión entre la app y la base de datos



Funciones clave de Supabase en Mas Tienda

Supabase Auth Registro, login y token JWT	PostgreSQL Tablas, relaciones, vistas e índices	Supabase Storage Imágenes de productos
RLS Seguridad por usuario y rol	API desde FastAPI Acceso controlado a los datos	

Función	Uso en Mas Tienda
Supabase Auth	Registro, login y token JWT
PostgreSQL	Tablas, relaciones, vistas e índices
Supabase Storage	Imágenes de productos
RLS	Seguridad por usuario y rol
API desde FastAPI	Acceso controlado a los dato

Normalización, restricciones e índices

1FN

Fotos, mensajes, costes e incidencias en tablas separadas. Sin listas en una sola columna.

2FN

Cada dato depende de la clave primaria completa de su tabla.

3FN

Sin datos repetidos. Pedidos no guarda el nombre del vendedor: lo obtiene por relación.

Restricciones

- **PRIMARY KEY** en todas las tablas
- **FOREIGN KEY** para conectar entidades
- **UNIQUE** en email, favoritos y conversaciones
- **ON DELETE CASCADE** en tablas dependientes
- **ON DELETE RESTRICT** en pedidos para proteger transacciones

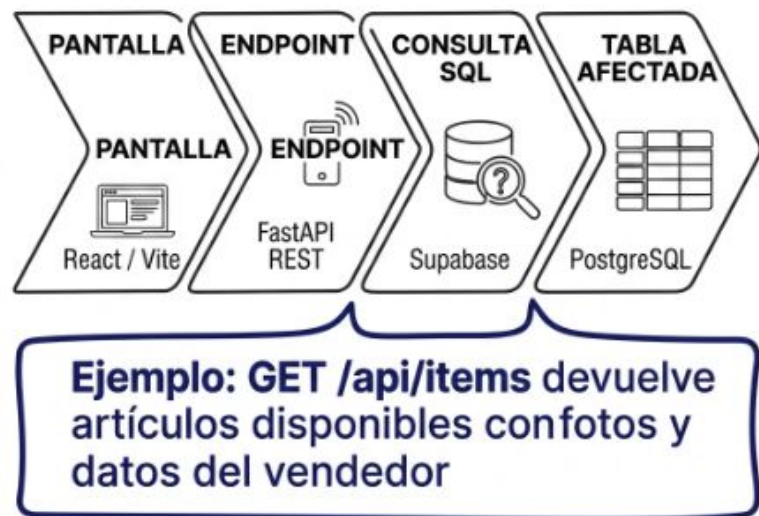
Índices clave

- `idx_articulos_estado_categoria_created`
- `idx_articulos_vendedor`
- `idx_ofertas_comprador_estado`
- `idx_pedidos_comprador_created`
- `idx_mensajes_conversacion_lectura`

«Normalizar no es decorar la base de datos; es evitar que el proyecto se rompa cuando crece.»

Aplicación y consultas empotradas

El frontend no ejecuta SQL directamente. **Axios** llama a **FastAPI**, que consulta **Supabase/PostgreSQL**.



Pantallas principales

- **Registro e inicio de sesión:** inserta o consulta usuarios
- **Página principal:** lista artículos disponibles
- **Detalle del artículo:** muestra producto, fotos y vendedor
- **Publicar producto:** inserta artículo y guarda URL de imagen
- **Favoritos:** inserta o elimina favoritos
- **Mensajes:** crea conversación y mensajes
- **Ofertas:** crea, actualiza o acepta ofertas
- **Perfil:** muestra productos, favoritos y compras
- **Administración:** gestiona usuarios, artículos y ofertas

Seguridad, IA y conclusión

Seguridad en tres capas

- **Supabase Auth:** autenticación con token JWT
- **Backend FastAPI:** autorización con `get_admin_user`, `verify_item_ownership`, `verify_offer_participation`
- **PostgreSQL:** Row Level Security con función `es_admin()`

Uso de IA

Apoyo para estructura, conexión frontend-backend, autenticación y diseño de pantallas. El equipo revisó y adaptó cada resultado al proyecto real.

Modelo E/R

Diseñado, normalizado y conectado

Integridad

Claves, restricciones e índices aplicados

Seguridad

Autenticación, autorización y RLS

«Sin una buena base de datos, Mas Tienda sería solo una interfaz bonita con problemas esperando explotar.»